ANANTA COMPANIES | INFORMATION & COMMUNICATION TECHNOLOGY

DocMan Alert Workflow & Architecture

How the system works, end to end

Components, data, roles, alert rules, scheduled jobs, deployment and operations. Version 2.0.

Document date	11 October 2026
Edition	2.0, replaces the 8 October 2026 edition
Audience	ICT, Compliance, system administrators
Classification	Internal

Contents

1. Purpose and scope . . . 5
2. System at a glance . . . 5
3. Architecture . . . 6
Blueprints . . . 6
4. Deployment . . . 6
5. Screens . . . 7
6. Backend modules . . . 8
7. Routes and JSON endpoints . . . 8
8. Data model . . . 10
9. Authentication and authorization . . . 10
10. Status engine . . . 11
11. Who receives an alert . . . 12
The matching rule . . . 12
Fixed mailboxes . . . 13
Worked example . . . 13
12. End-to-end workflows . . . 13
13. Notifications and alert history . . . 16
Failure handling . . . 16
Manual alert . . . 17
14. Scheduled jobs . . . 17
15. Files, backup and restore . . . 18
Backup . . . 18
Restore . . . 18
Lost Super Admin password . . . 18
16. OCR and field suggestion . . . 19
17. Excel import . . . 19
18. External services . . . 20
19. Security controls . . . 20

20. Configuration reference 21

21. Tests and quality 21

22. Operating notes and known limits 22

23. What this edition corrects or adds 22

Note
This edition was rewritten from the application source as it stands on 11 October 2026. Section 23 lists what differs from the 8 October edition. Where the code and an older document disagree, this document follows the code.

1. Purpose and scope

DocMan Alert is Ananta Companies' internal tracker for factory compliance documents, employee licenses and employee health checkups. It works out each record's expiry status in Bangladesh Standard Time, keeps the uploaded files, and emails the right people before something expires. It is one Flask application with one SQLite database, local file storage and one office mail relay.

This document is the technical companion to the User Manual. It describes how the pieces fit together, how a record moves from upload to alert, who is entitled to do what, and how the system is deployed and operated. It is written for ICT staff, system administrators and the Compliance team's owners.

- **Sections 2 to 8**: structure, deployment, screens, modules, routes and data.
- **Sections 9 to 13**: access control, status rules, who gets email and how mail is sent.
- **Sections 14 to 18**: scheduled jobs, files and backups, OCR, Excel import, integrations.
- **Sections 19 to 23**: security, configuration, tests, operating notes and edition history.

2. System at a glance

Users reach the application from a browser on the office network. The application stores records in SQLite, files on disk, reads scanned documents with a local OCR engine, and hands outgoing mail to the office relay. A scheduler (Windows Task Scheduler, or Kubernetes CronJobs in the container deployment) starts the background scripts that recalculate status, send alerts and take backups.

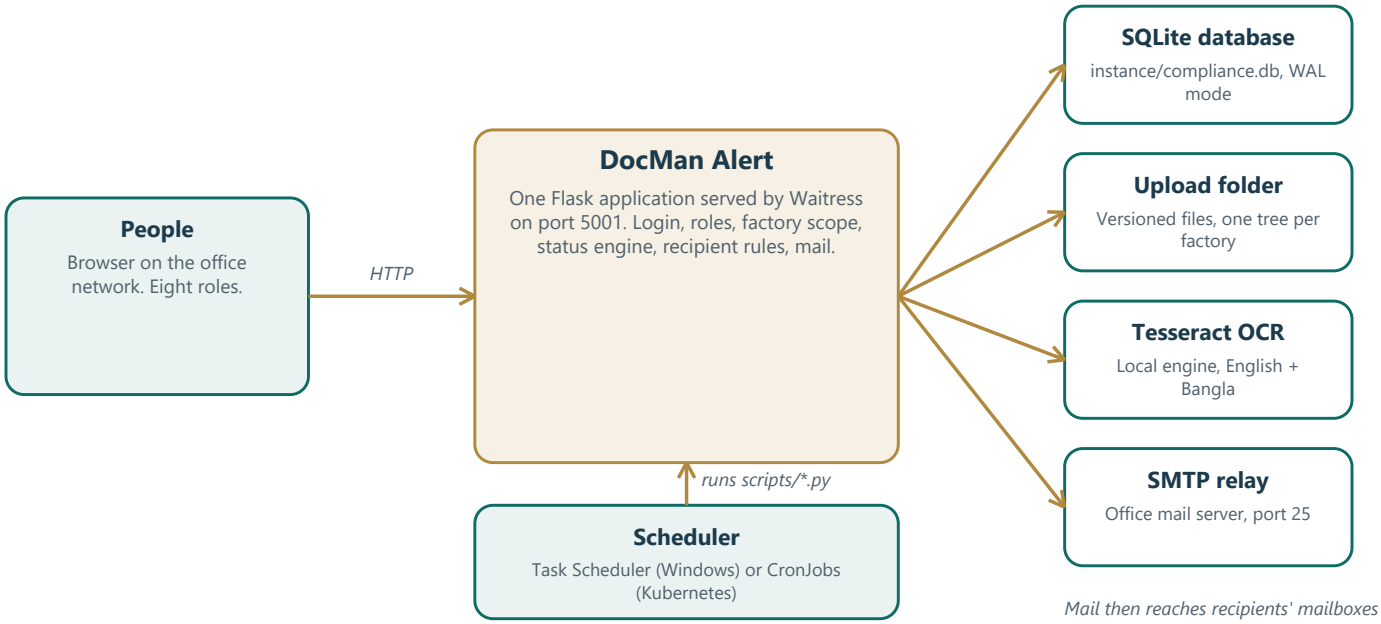


Figure 1. The system and the things it talks to.

Part	What it does
Browser	Renders server-built pages, posts forms with a CSRF token, downloads and previews files.
DocMan Alert	Flask application. Enforces login, role and factory scope, owns every business rule, writes the database and files.
SQLite database	instance/compliance.db. Organisation, people, records, alert history and logs.
Upload folder	app/uploads by default, or UPLOAD_FOLDER. Document, license and checkup files, versioned.
SMTP relay	192.168.10.6 port 25 by default. Messages are sent as document_tracker@ananta.com.bd.
Tesseract OCR	Local engine for scanned PDFs and images. No cloud OCR; files never leave the server.
Scheduler	Starts the server at boot, recalculates status daily, sends weekly mail, takes backups.

3. Architecture

The application is one process. There is no separate front-end build, message queue or microservice. A request reaches Waitress, Flask hands it to a blueprint, the route checks login, role and factory, calls the shared rules in app/utils.py, reads or writes SQLite through SQLAlchemy, and answers with a Jinja page, a file or a small JSON document. Uploaded files stay on disk and never in the database.

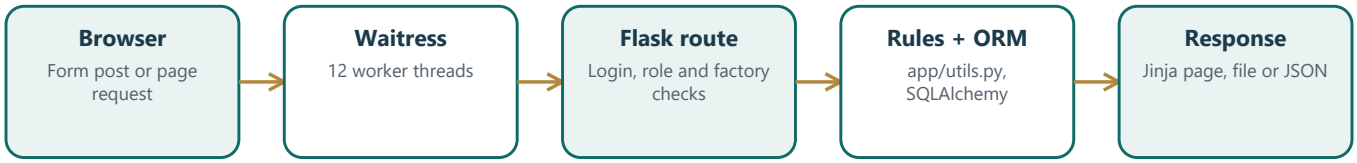


Figure 2. Path of a single request.

Layer	Technology	Responsibility
Pages	Jinja2 templates, Bootstrap (self-hosted), a little inline script	Screens, forms, dropdowns that refresh from JSON.
HTTP server	Waitress, 0.0.0.0:5001, 12 threads (serve.py)	Production process. run.py is development only.
Application	Flask 3 app factory, nine blueprints	Routes, validation, access control, mail.
Data	Flask-SQLAlchemy 3, SQLite in WAL mode	Tables, relationships, startup schema upgrades.
Files	Werkzeug uploads under UPLOAD_FOLDER	PDF, JPG, PNG, DOCX, XLSX, up to 10 MB.
Mail	Flask-Mail over SMTP	One message per recipient per record.
OCR	PyMuPDF and Tesseract (pytesseract)	Suggests field values. Never saves a record by itself.
Reading Excel	openpyxl	Import preview and confirm.

Blueprints

Blueprint	URL prefix	Contains
auth	`/login, /logout`	Sign-in, lockout, sign-out.
main	`/, /my-alerts`	Dashboard and My Alerts.
documents	`/documents`	Factory documents: list, add, edit, renew, archive, delete, files, OCR, send alert, JSON helpers.
employees	`/employees`	Employees, licenses, health checkups, OCR, Excel import.
alerts	`/alert-history`	Sent and failed alert email history.
admin	`/admin`	Companies, factories, departments, recipient pickers, backups.
users	`/users`	User accounts.
regulatory	`/regulatory`	Category, authority and service lists and their JSON helpers.
logs	`/system-logs`	System Logs (Super Admin).

Startup (create_app) loads config.py, opens SQLite with WAL, a 5-second busy timeout and foreign keys on, creates missing tables, runs the idempotent upgrade in app/schema_upgrade.py, registers the blueprints, security headers and error pages, clears abandoned OCR staging files and writes an APP_START log line.

4. Deployment

The same code runs in two supported arrangements. Both keep exactly one application instance, because SQLite allows one writer.

	Windows host (original)	Container and Kubernetes (deploy/)
Runtime	venv with Python 3.14, Tesseract installed on the host.	python:3.12-slim image with Tesseract eng + ben, tzdata, tini.
Start	Task Scheduler task at startup runs scripts/run_server.bat, which runs serve.py.	Deployment 'compliance-tracker', one replica, strategy Recreate, command python serve.py.
Schedule	Three more Task Scheduler tasks, created by scripts/setup_task_scheduler.bat.	Three CronJobs in namespace compliance, timeZone Asia/Dhaka, concurrencyPolicy Forbid.
Data	instance/compliance.db, app/uploads, backups/ under the project folder (or the *_FOLDER variables).	PVC compliance-data (8 Gi, local-path) mounted at /data. Entry script seeds it on first boot only.
Secrets	Machine-level environment variables (setx /M). The tasks run as SYSTEM, which does not see a user's variables.	Kubernetes Secret compliance-secrets, loaded with envFrom.
Network	http://host:5001. Optional internal reverse proxy for HTTPS.	Service port 80 to 5001, Traefik Ingress on web and websecure, probes on /login.

Important
Never run more than one application instance against the same database file. The Deployment uses **replicas: 1** and **strategy: Recreate** so two pods can never overlap during a rollout.

In the container the SQLite path is fixed by config.py to /app/instance, so the entry script replaces that folder with a link to /data/instance. UPLOAD_FOLDER and BACKUP_FOLDER point at /data. Status calculations use a fixed UTC+6 offset in code, so they do not depend on the container clock; TZ=Asia/Dhaka is set so log timestamps and CronJob times read as Bangladesh time.

5. Screens

Every screen is server-rendered HTML built from templates/base.html and one page template. There is no single-page application and no client-side data store; the browser keeps only the session cookie. The left menu shows only what the signed-in role may open.

Screen	Who	What happens there
Login	Everyone	Username and password. A locked account is told how long to wait.
Dashboard	Everyone, factory scoped	Counts by status for documents and for employee licenses and checkups; factory table; up to 12 urgent items.
My Alerts	Everyone	Records that will really email this user, and why.
Alert History	Everyone, factory scoped	Every alert email handed to the mail server, sent or failed.
Documents	Everyone, factory scoped	Search and filter; add, edit, renew, extract from file, archive and restore, delete, send alert.
Employee Compliance	Everyone, factory scoped	Employees; Employee Licenses and Health Checkups lists; add or edit records; extract from file; archive.
Import	Super Admin	Excel preview then confirm.
Companies	Super Admin, Administration	Company list and company alert recipients.
Factories	Super Admin	Factory list, add, edit, delete when unused.
Departments	Super Admin, Administration, Factory Admin	Shared department list and group mailboxes.
Dept. Alert Recipients	Super Admin, Administration, Factory Admin	Recipients for one factory and department.

Screen	Who	What happens there
Users	Super Admin, Factory Admin	Accounts, roles, factory, department, status.
Regulatory Directory	Super Admin	Categories, authorities, services.
Backups	Super Admin	Create and download backup zips.
System Logs	Super Admin	Operational events with level and type filters.

Four small fetch calls refresh dropdowns without reloading the page: the department list on the document form, the matching users and group mailboxes for a factory and department, and the authority and service lists of the regulatory chain.

6. Backend modules

Module	Role
`app/__init__.py`	App factory, extensions, security headers, error pages, startup log.
`app/routes/*.py`	One file per blueprint. Each function checks login, role and factory.
`app/forms.py`	WTForms validation for login, documents, employees, users, organisation and regulatory records.
`app/models.py`	SQLAlchemy tables and relationships.
`app/utils.py`	BST time helpers, roles_required, status engine, recipient resolution, mail building and sending, alert history, system log writer.
`app/ocr_service.py`	Reads a PDF or image on a temporary copy and suggests fields. Aborts if the source file changes.
`app/import_utils.py`	Parses and validates Excel rows. Writes nothing until the user confirms.
`app/backup_service.py`	Consistent SQLite snapshot plus the upload tree into a timestamped zip; retention.
`app/schema_upgrade.py`	Startup upgrade: new columns and indexes, department de-duplication, seed of Transport roles, Head Office, Compliance and Transport departments. Never drops data.
`scripts/`	Scheduled jobs, backup and restore, one-off migrations, the regulatory data seeder, PDF builders.

Business rules live in one place. `compute_status()` is the only source of status and `resolve_alert_recipients()` is the only source of who is emailed. Routes, jobs and the My Alerts page all call them, so screens and email can never disagree.

7. Routes and JSON endpoints

The application is form-driven. All state-changing routes are POST with a CSRF token; downloads are GET routes that check factory scope first. MANAGE means Super Admin, Administration, Factory Admin, Compliance Manager, Compliance User, Transport Manager and Transport User. DELETE means Super Admin, Administration, Factory Admin, Compliance Manager and Transport Manager.

Route	Access	Purpose
`GET /documents/`	Signed in	List with search, factory, department, status, type and expiry-range filters; 10 per page; ?archived=1.
`GET, POST /documents/add`	MANAGE	Create a document and its first file version.
`GET, POST /documents/edit/<id>`	MANAGE	Edit; a new file becomes the next version.
`GET, POST /documents/renew/<id>`	MANAGE	New dates and file; marks renewal Completed.

Route	Access	Purpose
`POST /documents/archive, /restore/<id>`	DELETE	Hide or bring back a document and keep its files.
`GET, POST /documents/delete/<id>`	DELETE	Permanent delete after typing the exact name.
`GET, POST /documents/send-alert/<id>`	MANAGE	Manual alert to ticked users and group mailboxes.
`POST /documents/extract; GET, POST /documents/extract/<token>`	MANAGE	OCR a new upload, then confirm to create the document.
`POST /documents/ocr/<version>; GET, POST .../view`	MANAGE (view: any)	Run OCR on a stored file; review and apply fields.
`GET /documents/download, /file/<version>`	Signed in, in scope	Download or preview one file version.
`GET /documents/api/alert-users`	Signed in, in scope	Users and group mailboxes for a factory and department.
`GET /documents/api/departments/<factory>`	Signed in	Shared department list; the factory id is ignored.
`GET /employees/, /licenses, /checkups`	Signed in, in scope	Employee list; cross-employee license and checkup lists with search.
`GET, POST /employees/add, /edit/<id>`	MANAGE	Employee record. No delete; set Inactive.
`GET, POST /employees/<id>/license/add, /checkup/add`	MANAGE	Add a license or health checkup (optionally extracted from a file).
`GET, POST /employees/license|checkup/<id>/edit`	MANAGE	Edit; a new file becomes the next version.
`POST /employees/license|checkup/<id>/archive`	DELETE	Hide a record. There is no restore screen.
`GET, POST /employees/import, POST /import/confirm`	Super Admin	Excel preview and confirm.
`GET /my-alerts`	Signed in	Records that will email the current user.
`GET /alert-history/, /<id>`	Signed in, in scope	History list (30 per page) and detail.
`GET, POST /users/, /add, /edit/<id>, /delete/<id>`	Super Admin, Factory Admin	Accounts. Factory Admin limited to own factory.
`GET, POST /admin/companies ...`	Super Admin (list and recipients: + Administration)	Companies.
`GET, POST /admin/factories ...`	Super Admin	Factories.
`GET, POST /admin/departments ...`	Super Admin, Factory Admin (view: + Administration)	Departments and group mailbox.
`GET, POST /admin/department-recipients`	Super Admin, Administration, Factory Admin	Recipients for a factory and department pair.
`GET, POST /admin/backups, GET /backups/download/<file>`	Super Admin	Create and fetch backup zips.
`/regulatory/categories|authorities|services ...`	Super Admin	Regulatory lists.
`GET /regulatory/api/authorities|services/<id>`	Super Admin, Administration	Cascading dropdowns on the document form.
`GET /system-logs/`	Super Admin	Log list with level and event filters.

Table 7. Routes. JSON endpoints return data only for a signed-in session; there is no public API.

8. Data model

SQLite stores operational data in instance/compliance.db with write-ahead logging, so a reader can continue while a status update or backup writes.

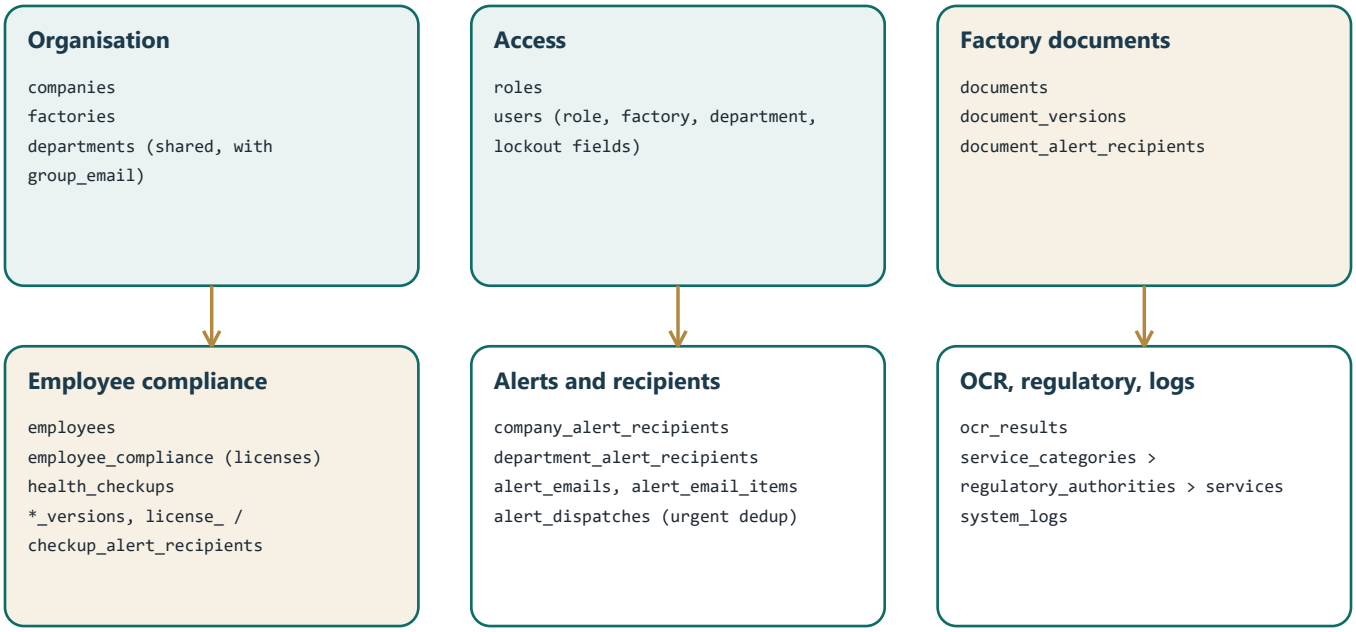


Figure 3. Table groups. Arrows point from a group to the group that depends on it.

Group	Tables and notes
Organisation	**companies** hold **factories**. **departments** are one shared list for the whole company (unique name, optional group_email); a department is not owned by a factory.
Access	**roles** and **users**. A user has one role, an optional factory (empty means Global Access) and an optional department. failed_login_attempts and locked_until persist the lockout.
Factory documents	**documents** (factory, department, optional category, authority, service, creator, status, no_expiry, is_active) with **document_versions** (file path, issue and expiry per version) and **document_alert_recipients**.
Employee compliance	**employees** (unique code per factory, department, personal email, receive_alerts, is_active), **employee_compliance** for licenses, **health_checkups**, their file-version tables and **license_alert_recipients**, **checkup_alert_recipients**.
Alerts	**company_alert_recipients** and **department_alert_recipients** link users to an organisation. **alert_emails** and **alert_email_items** are the send history. **alert_dispatches** stops a repeat urgent email for the same record, status and expiry date.
OCR, regulatory, logs	**ocr_results** keeps extracted text and suggested fields, one per file. **service_categories** to **regulatory_authorities** to **services**. **system_logs** keeps operational events only.

- Recipient links always point at users.email; no address is copied into a link table.
- A no-expiry record stores expiry_date empty together with no_expiry set. Placeholder text is never stored in a date.
- Deleting a user clears that user's created_by and uploaded_by references and removes recipient links; records are kept.
- Stored times are UTC. Screens convert them to Bangladesh time with to_bst().

9. Authentication and authorization

Sign-in checks the password against a Werkzeug hash. Five wrong passwords lock the account for 10 minutes; the lock is stored on the user row and survives a restart. A successful sign-in clears it. Sign-out is a POST. The session cookie is HttpOnly, SameSite=Lax, Secure when SESSION_COOKIE_SECURE is on, and not permanent, so it ends when the browser closes; there is no idle timeout. The process refuses to start without SECRET_KEY.

Visibility has two layers. A **role** decides which actions exist. A **factory scope** decides which records are visible: Super Admin, Administration and Compliance Viewer see every factory; every other role sees only the factory on its account. A factory-scoped user who asks for another factory's record, file or form receives 403.

Capability	SA	ADM	FA	CM	CU	CV	TM	TU
See every factory's records	Yes	Yes	-	-	-	Yes	-	-
View documents, employees, alert history	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Add, edit, renew, extract, send alert	Yes	Yes	Yes	Yes	Yes	-	Yes	Yes
Archive, restore, delete documents	Yes	Yes	Yes	Yes	-	-	Yes	-
Add or edit employees, licenses, checkups	Yes	Yes	Yes	Yes	Yes	-	Yes	Yes
Archive a license or checkup	Yes	Yes	Yes	Yes	-	-	Yes	-
Link a document to a regulatory service	Yes	Yes	-	-	-	-	-	-
Set company alert recipients	Yes	Yes	-	-	-	-	-	-
Set department recipients; view departments	Yes	Yes	Yes	-	-	-	-	-
Add, edit, delete departments	Yes	-	Yes	-	-	-	-	-
Manage users (Factory Admin: own factory)	Yes	-	Yes	-	-	-	-	-
Companies, factories, regulatory lists, import, backups, logs	Yes	-	-	-	-	-	-	-

Table 9. SA Super Admin, ADM Administration, FA Factory Admin, CM Compliance Manager, CU Compliance User, CV Compliance Viewer, TM Transport Manager, TU Transport User. Transport roles have the same rights as the matching Compliance roles.

A Factory Admin can create users only in their own factory and only with the roles Compliance Manager, Compliance User, Compliance Viewer, Transport Manager or Transport User. Only a Super Admin can change a user's factory. Nobody can change their own role or active status. Users are deleted by typing the username; this is blocked for one's own account, for the protected default *admin* account and for the last Super Admin. The protected account allows only an email change.

10. Status engine

Dates are compared in Bangladesh Standard Time (UTC+6) by one function. Documents, employee licenses and health checkups all use it. A record with no expiry date, or with no_expiry set, is Valid.

Days until expiry	Status	What it triggers
More than 90, or no expiry	Valid	No expiry email.
61 to 90	Expiring in 90 Days	Weekly email (Saturday 09:00).
31 to 60	Expiring in 60 Days	Weekly email (Saturday 09:00).
8 to 30	Expiring in 30 Days	Weekly email (Saturday 09:00).
1 to 7	Expiring in 7 Days	Urgent email at the next 09:00 run, once for this status and expiry date.
0 or earlier	Expired	Urgent email at the next 09:00 run, once for this status and expiry date.

Status is recalculated when a record is saved, when the dashboard opens, at the start of the weekly job and by the daily job, using set-based UPDATE statements. Opening the dashboard writes status values but never sends mail. The email itself always shows a status recomputed from the expiry date, so a stale stored value cannot put the wrong status in a message.

11. Who receives an alert

Every automatic alert, the My Alerts page and the recipient list on a document's detail page come from one function, `resolve_alert_recipients()`. It first picks a group of people, then keeps only those who belong to the record's factory and department, then adds a few fixed mailboxes.



Figure 4. How recipients are chosen for one record.

The matching rule

A user is a candidate only when the account is active, has a valid email address, and matches the record's factory and department. A user matches when the user's factory is empty (Global Access) or equals the record's factory, **and**, if the record has a department, the user's department equals it. A user with no department therefore does not match any record that has one, whatever the role. This applies to assigned people, department and company recipients and the uploader alike.

Fixed mailboxes

Source label	Added when	Configured in
Production	The record's factory code is HO (Head Office or any name containing it): arifkhan@ and badhon_admin@ananta.com.bd.	PRODUCTION_ALERT_ROUTES in config.py
Department group	The record's department has a group_email, or is named Compliance or Transport (compliance_dept@, transport_agl@).	Department row; DEPARTMENT_GROUP_EMAILS fallback
Department group (always)	Every record, whatever its factory or department: compliance_dept@ananta.com.bd. Compliance is a global watcher by policy.	ALWAYS_INCLUDE_GROUP_EMAILS
Uploader	The record's creator, when active, valid and matching.	Record created_by
Employee	License or checkup whose employee is active, has receive_alerts on and a valid email.	Employee record
Document-specific / Record-specific	Users ticked on the record.	Document, license or checkup
Department / Company	Fallback layers above.	Departments and Companies screens

Note
A record with no assigned person, no department recipient and no company recipient still emails the compliance group mailbox, because that address is always included. Alert History shows the source of each message so an unexpected recipient can be traced.

Worked example

A Fire License belongs to Ananta Garments Ltd., department Compliance. Farhana, a Compliance Manager in that department, uploaded it and is also ticked on the document. Imran is a recipient of the Compliance department. The result is: Farhana (Document-specific and Uploader) and compliance_dept@ (Department group). Imran is **not** emailed, because a person was assigned on the record and the department layer is skipped. If nobody were ticked, Imran would receive it as a Department recipient.

12. End-to-end workflows

Five workflows cover almost everything the application does. Each diagram is a path from a user action to a stored result.

12.1 Document or record creation

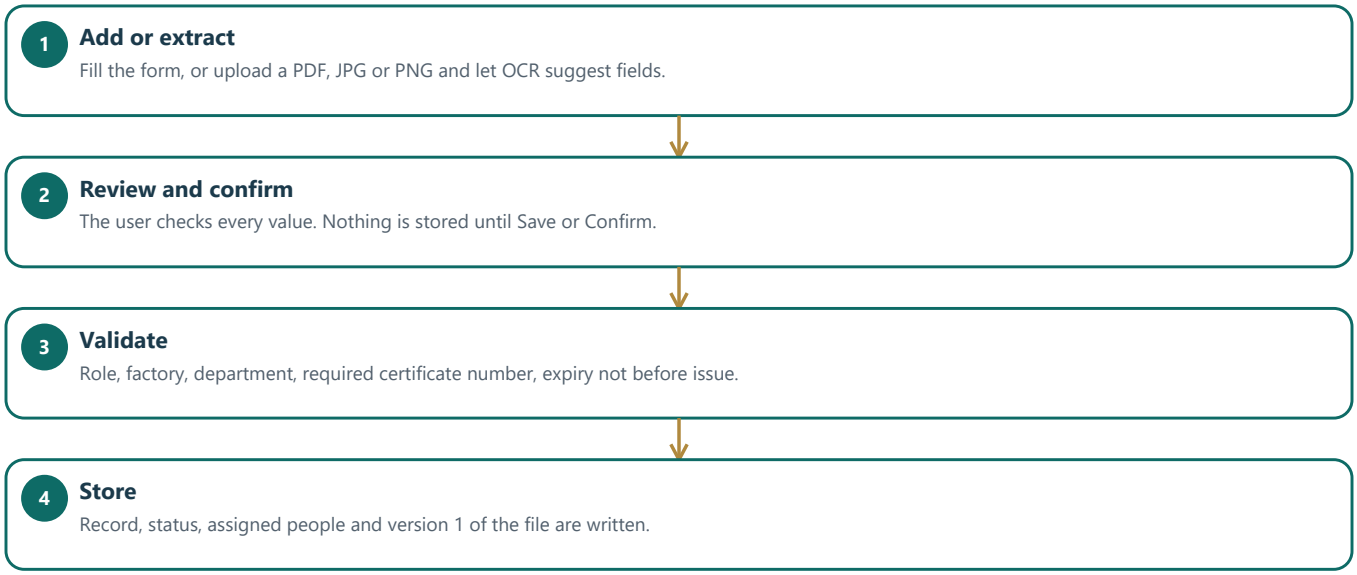


Figure 5. A record becomes stored data.

12.2 Renewal

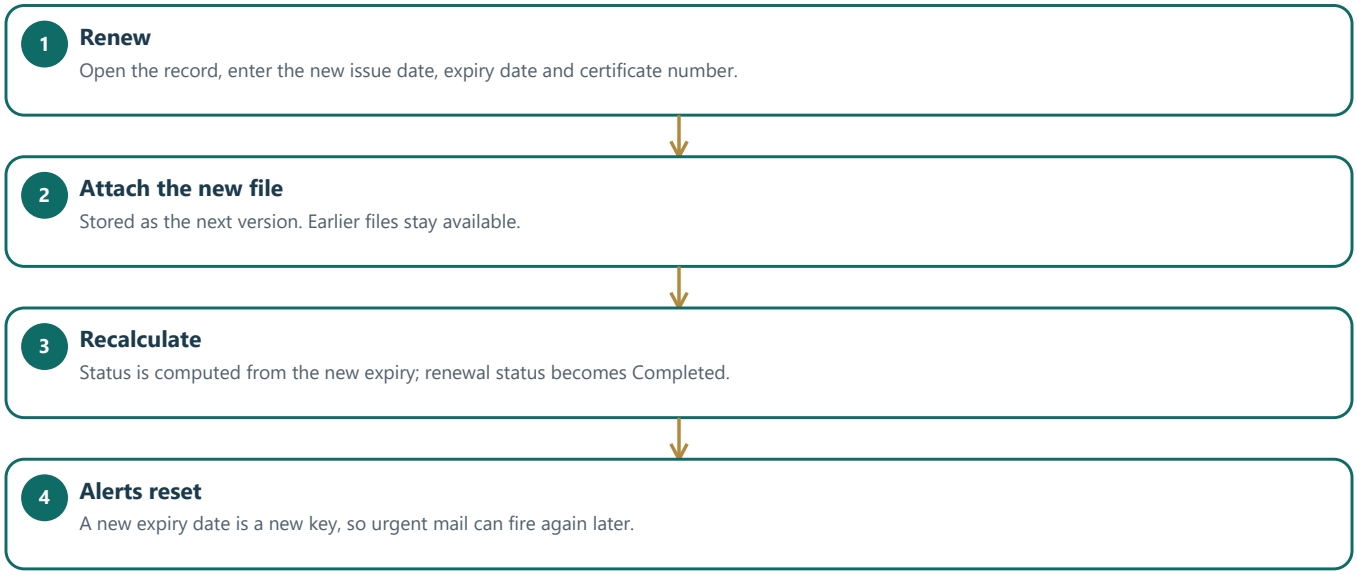


Figure 6. Renewal keeps history. For licenses and checkups the same effect comes from editing and uploading a new file.

12.3 Expiry monitoring and alert

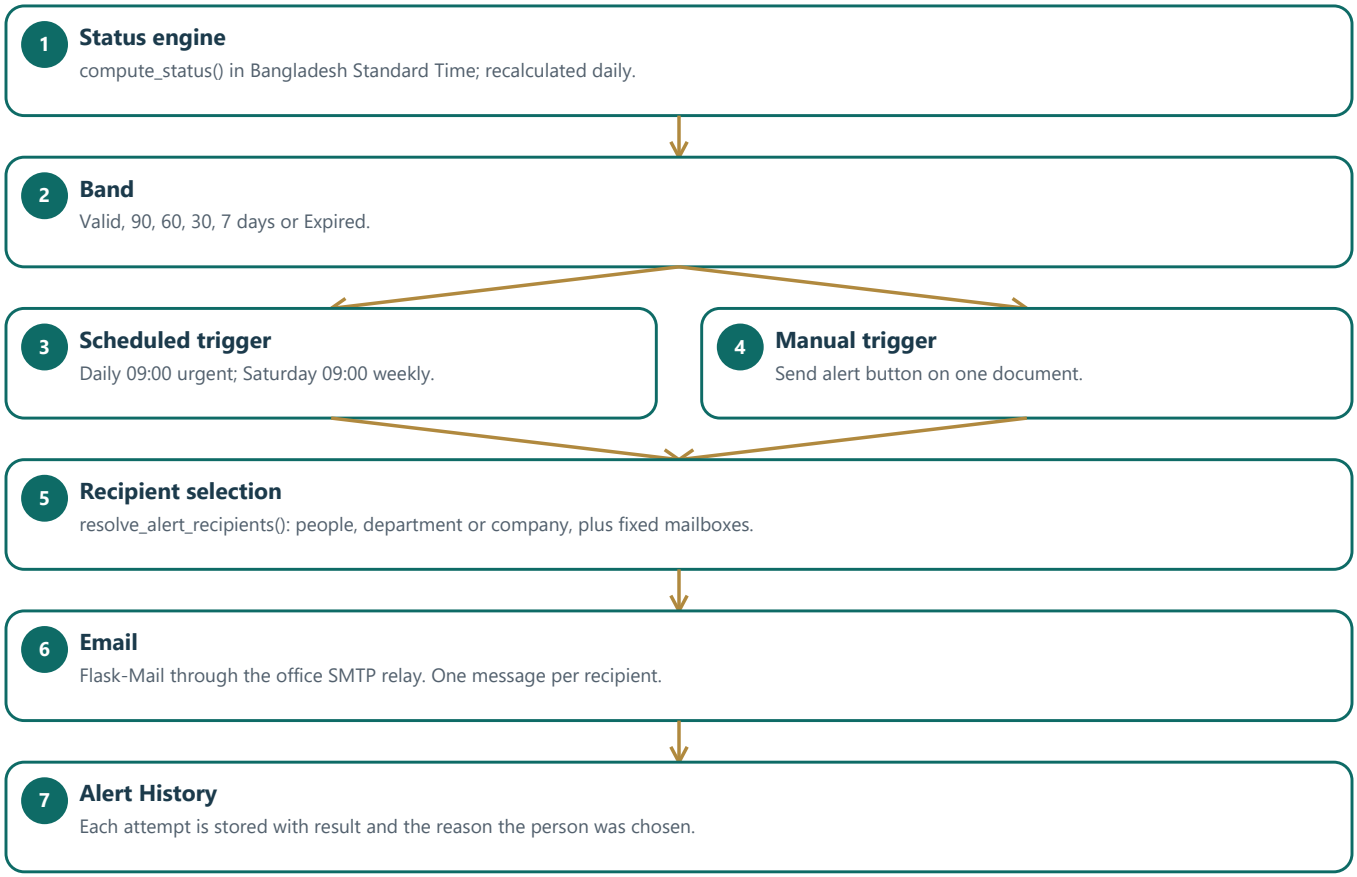


Figure 7. From a date to a delivered, recorded email.

12.4 Extract from a file

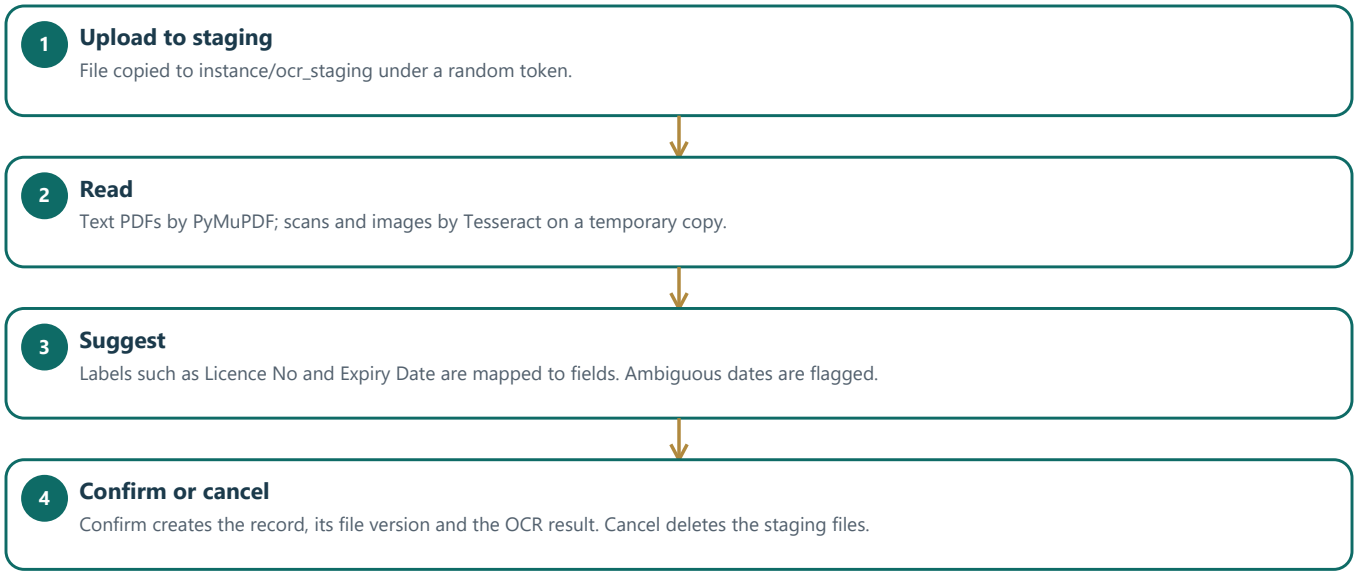


Figure 8. OCR only suggests. The user owns every saved value.

12.5 Excel import

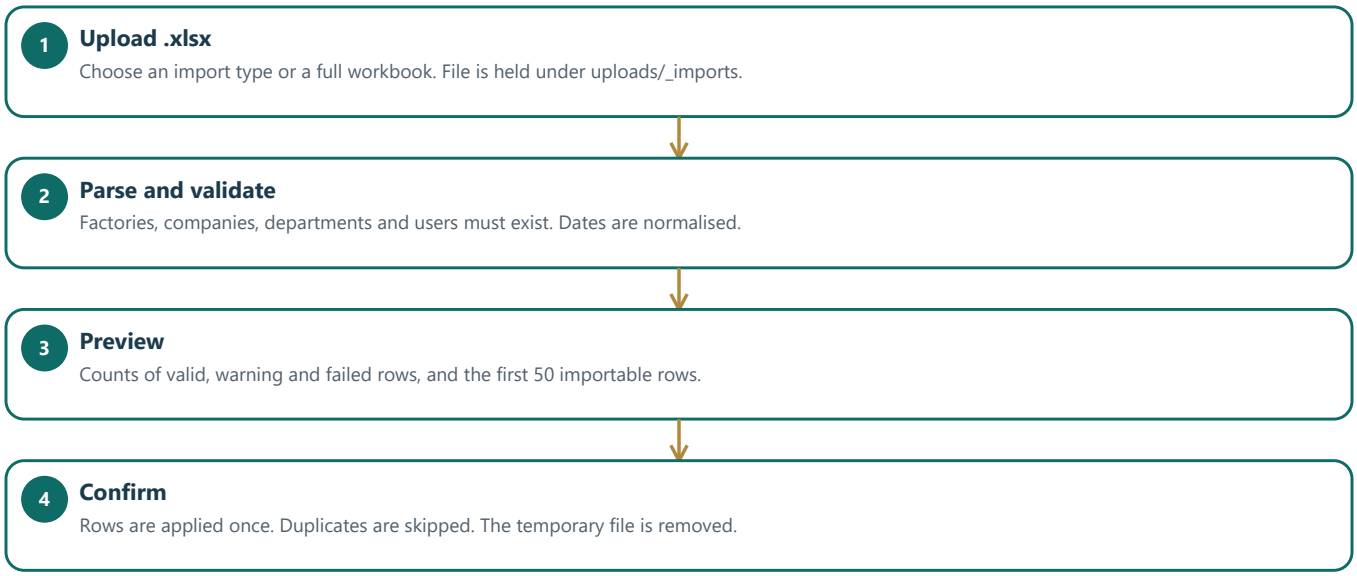


Figure 9. Super Admin only. Nothing is created automatically except the imported records.

13. Notifications and alert history

There is no in-app inbox. Notification means email, plus the Alert History page. Each email is about exactly one record, so a recipient with five expiring records receives five messages.

Type	When	Subject	Repeat control
Weekly (weekly)	Saturday 09:00, for 90, 60 and 30 day records	Compliance Alert: <title> - <status>	Sent every week while the status holds.
Urgent (emergency)	Daily 09:00, for 7 day and Expired records	URGENT: <title> Expires in N Days, or Expired	Once per record, status and expiry date (table alert_dispatches).
Manual (manual)	When someone uses Send alert on a document	Compliance Notice: <title>	None. Does not stamp the urgent dedup row.

The body lists Document, Document Type, Certificate or License No., Company, Factory, Department, Issue Date, Expiry Date, Remaining and Alert Status; empty optional rows are left out. The same layout is used for weekly, urgent and manual mail. Dates print as 15-Oct-2026.

Failure handling

- Messages go one at a time, so one bad address does not stop the rest. Each attempt is stored in alert_emails with result sent or failed and the exception class name (never the SMTP text).
- An urgent run stamps the dedup row for a record when at least one message for it was delivered. A record whose every message failed is tried again at the next run. A recipient who failed while others succeeded is not retried for that status.
- The weekly run loads at most 200 documents, 200 licenses and 200 checkups, earliest expiry first.
- Skipped addresses (invalid or missing email) are written once per run as ALERT_RECIPIENT_SKIPPED.

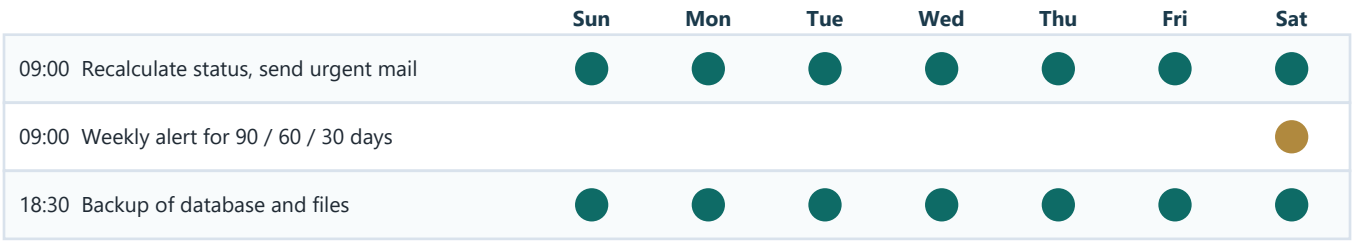
Alert History is filtered to the factories a user may see. Test messages (type test) are visible only to Super Admin; the Test Alert screen itself no longer exists, so only older rows carry that type.

Manual alert

The Send alert page lists the users in the document's factory and department and the department's group mailboxes. Group mailboxes are ticked by default. Company recipients, the uploader and Head Office mailboxes are not added. **The people ticked are also saved as the document's assigned recipients**, which replaces the earlier assignment and changes who receives later automatic alerts.

14. Scheduled jobs

Four things run on a timetable. On Windows, scripts/setup_task_scheduler.bat creates four tasks that run as SYSTEM; in the container deployment the same scripts run as three Kubernetes CronJobs next to the always-on web pod.



All times are Bangladesh Standard Time (UTC+6).

Figure 10. A normal week.

Task	When	Script	Effect
ComplianceTracker_Server	At startup	run_server.bat, serve.py	Waitress on port 5001.
ComplianceTracker_StatusRecalc	Daily 09:00	scheduled_status_recalc.py	Recalculate every status, then send urgent emails for 7 day and Expired records.
ComplianceTracker_EmailAlert	Saturday 09:00	scheduled_email_alert.py	One email per record per recipient for the 90, 60 and 30 day bands.
ComplianceTracker_Backup	Daily 18:30	backup.py	Zip database and uploads; mirror; delete copies past retention.

CronJob	Schedule (Asia/Dhaka)	Command
compliance-backup	`30 18 * * *`	`python scripts/backup.py`
compliance-status-recalc	`0 9 * * *`	`python scripts/scheduled_status_recalc.py`
compliance-weekly-alert	`0 9 * * 6`	`python scripts/scheduled_email_alert.py`

Each job writes JOB_START and JOB_SUCCESS or JOB_FAILURE to System Logs and exits 0 on success, 1 on failure. Setting COMPLIANCE_SKIP_EMERGENCY=1 makes the daily job recalculate without sending mail; tests use it. The other files in scripts/ are one-off, hand-run tools: schema migrations, data repairs, the regulatory data importer and restore.py.

15. Files, backup and restore

What	Where (under UPLOAD_FOLDER unless stated)	Stored name
Document file	`factory_<factory id>/`	`doc<id>_v<version>_<file name>`
License file	`employees/factory_<factory id>/`	`lic<id>_v<version>_<file name>`
Checkup file	`employees/factory_<factory id>/`	`chk<id>_v<version>_<file name>`
Excel import in progress	`_imports/`	`<token>.xlsx`
OCR extraction in progress	`instance/ocr_staging/ (not under UPLOAD_FOLDER)`	`<token>.pdf|jpg|png and <token>.json`

An upload is checked for extension (pdf, jpg, jpeg, png, docx, xlsx) and size (10 MB). The file name is passed through secure_filename and the storage name is generated by the server; the database holds the relative path. Download and delete resolve the real path and refuse anything outside the upload folder.

Archive hides a document and keeps every file; it can be restored from View Archived. **Permanent delete** asks for the exact document name, then removes the document, its versions, recipient links, urgent-dedup rows and files. Alert History keeps the entries for mail already sent. Licenses and checkups can be archived but not deleted or restored from the screens; an employee is made Inactive instead of deleted.

Backup

A backup is a zip named compliance_backup_YYYY-MM-DD_HH-MM-SS.zip containing compliance.db, taken with the SQLite backup API so it is consistent while the app is running, and the whole upload tree under uploads/. If BACKUP_MIRROR_FOLDER is set a second copy is written there. Copies older than BACKUP_RETENTION_DAYS (default 30) are removed from both places. A Super Admin can also create one from the Backups screen and download any backup in the primary folder.

Restore

1. Stop the application. In Kubernetes, scale the Deployment to 0 and run the script in a one-off pod that mounts the same volume.
2. From the project folder run `venv\Scripts\python.exe scripts\restore.py`.
3. Pick the backup number from the list and type **YES** to confirm.
4. The script copies the current database and uploads to a folder named pre_restore_safety_<date-time>, extracts the backup, checks that the core tables exist, replaces the database and uploads, and writes RESTORE_SUCCESS or RESTORE_FAILURE to System Logs.
5. Start the application again with the normal production method (serve.py, the scheduled task or the Deployment), sign in and check Documents and Users.
6. Delete the safety folder when you are satisfied.

> **Important**
> RESTORE_GUIDE.txt (in Bangla) still tells the operator to start the app with **python run.py**. That is the development server. Use serve.py or the scheduled task in production. A backup only proves recoverability when a restore has been tried; rehearse one on a scratch copy.

Lost Super Admin password

reset_admin_password.py (project root) prompts for the new password without echo, requires at least 10 characters with an upper-case letter, a lower-case letter and a digit, asks for YES and updates only the *admin* account. No password is stored in a file.

16. OCR and field suggestion

OCR is optional and local. If Tesseract is missing or OCR_ENABLED is off the rest of the application works and OCR screens say it is unavailable.

Step	Detail
Accepted files	PDF, JPG, JPEG, PNG. At most OCR_MAX_FILE_MB (10) and OCR_MAX_PDF_PAGES (50).
Text PDFs	Embedded text is read by PyMuPDF when it has at least 40 letters or digits.
Scans and images	Pages rendered at 2x, converted to grayscale, widened to 1400 px if smaller, contrast stretched; Tesseract with OCR_LANGUAGES (eng+ben), OCR_TIMEOUT_SECONDS (120).
Safety	Works on a temporary copy. Stops if the original's size or modified time changes.
Confidence	Average of word confidences. Below 60 shows a warning to check the values.
Suggested fields	Labels such as Certificate No, License No, Registration No, Issued by, Issue Date, Expiry Date or Valid until, Report Date, Test Center, Company, Department are read from the same line or the line after.
Dates	12 Jan 2026 and 2026-01-12 are accepted. 05/06/2026 is ambiguous and is never applied; 13/06/2026 is read as day first. Bangla digits are converted.
Search	Extracted text is stored, with a digit-normalised copy, and is searched together with the record fields in the document, license and checkup lists.

17. Excel import

Super Admin only (Employee Compliance, Import). Choose a type, upload an .xlsx file, read the preview and confirm. Companies, factories, departments and users must already exist; unknown ones are reported as failed rows and are never created.

Import type	Required columns	Optional columns
Employees	Employee ID, Employee Name, Factory	Designation, Section, Company, Department, Date of Joining, Email
Employee Licenses / Medical Licenses	Name, ID No (or Employee ID), Factory	Designation, Section, Registration No, Category, Issue Date, Expiry Date, Remarks, Company, Department, License Category
Health Checkups	Employee ID, Name, Factory	Section, Designation, DOJ, PRE Y/N, Test Center, Company, Department, Remarks, and report and expiry columns for CBC, Hearing / Widal, Chest X-Ray, Urine Test
Alert Recipients	Company, Recipient Scope (COMPANY or DEPARTMENT), Username or Email	Factory; Department is required for scope DEPARTMENT
Full Workbook	Sheets named Employees, Employee Licenses, Medical Licenses, Health Checkups, Alert Recipients	Sheets are imported in that order; other sheets are ignored.

- Dates may be Excel serial numbers, 2026-01-12, 12-01-2026, 12/01/2026, 12-Jan-2026 or with Bangla digits. Values such as Till Now, N/A, Permanent or a dash become *no expiry*.
- Duplicates are skipped, not overwritten: employee by factory and ID, license by employee, type and registration number, checkup by employee, exam and report date, recipient link by user and organisation.
- Factories are matched by name or by the capital letters of the name (AGL, ASL, ...).
- Imported licenses and checkups have no uploader, no assigned people and no file.

18. External services

Service	Used for	What crosses the boundary
Office SMTP relay 192.168.10.6:25	Weekly, urgent and manual alerts	HTML mail from document_tracker@ananta.com.bd. No TLS on the default relay.
Tesseract OCR	Reading scans and images	Image in, text out, on the same server.
PyMuPDF	Reading text PDFs and rendering pages	Local library, no network.
Windows Task Scheduler / Kubernetes CronJobs	Server start, jobs	Starts a local Python process.
Excel workbooks	Import	Parsed in memory; preview first.

There is no cloud storage, external identity provider or third-party notification service. Identity is the users table; delivery is the office mail relay.

19. Security controls

- Session, role and factory are checked on every business page, action and file. Hiding a menu item is a convenience; the server enforces access.
- CSRF token on every form post including sign-out. Passwords are stored only as hashes.
- Account lockout after five failures, stored in the database. Same message for unknown user and wrong password.
- Response headers: no-store caching outside /static, X-Content-Type-Options nosniff, X-Frame-Options SAMEORIGIN, Referrer-Policy same-origin, and HSTS when SESSION_COOKIE_SECURE is on.
- Upload type and size limits, server-generated names, downloads confined to the upload folder.
- Permanent delete needs the exact record name and a role allowed to delete.
- Error pages show no stack trace. System Logs hold operational events, never document contents or passwords.
- SECRET_KEY and the SMTP password come only from the environment.

20. Configuration reference

Variable	Default	Meaning
SECRET_KEY	none (required)	Signs the session. The process will not start without it.
UPLOAD_FOLDER	app/uploads	Where uploaded files are stored.
BACKUP_FOLDER	backups/	Primary backup folder.
BACKUP_MIRROR_FOLDER	none	Optional second copy; use another disk or share.
BACKUP_RETENTION_DAYS	30	Age after which backups are removed.
SMTP_HOST, SMTP_PORT	192.168.10.6, 25	Mail relay.
SMTP_USE_TLS, SMTP_USE_SSL	0, 0	Transport security.
SMTP_USERNAME, SMTP_PASSWORD (or MAIL_PASSWORD)	document_tracker@ananta.com.bd, none	Relay login. Password has no default.
SMTP_SENDER	document_tracker@ananta.com.bd	From address of alerts.
ALWAYS_INCLUDE_GROUP_EMAILS	compliance_dept@ananta.com.bd	Comma or semicolon list copied on every alert.
SESSION_COOKIE_SECURE	0	Set to 1 once served over HTTPS.
OCR_ENABLED	1	Turn OCR off or on.
TESSERACT_CMD, TESSDATA_PREFIX	Windows default path; project tessdata/	Local OCR engine and language data.
OCR_LANGUAGES	eng+ben	Tesseract languages.
OCR_MAX_FILE_MB, OCR_MAX_PDF_PAGES, OCR_TIMEOUT_SECONDS	10, 50, 120	OCR limits.

PRODUCTION_ALERT_ROUTES and DEPARTMENT_GROUP_EMAILS are set in config.py and need a code change and restart. The ALERT_RECIPIENTS variable mentioned in .env.example is legacy and is not read.

21. Tests and quality

tests/run_all.py runs each suite in its own process without pytest. The 8 October 2026 QA pass (QA_TEST_REPORT.md) recorded all suites green and found no critical or high defects. The suites write and remove their own ZZ-prefixed fixtures in the configured database, so run them against a copy, not the live installation.

Suite	Covers
test_status_engine	Status boundaries at 0, 1, 7, 8, 30, 31, 60, 61, 90, 91 days and no expiry.
test_routes_smoke, test_isolation_rbac	Pages load; factory isolation and Super Admin only screens.
test_alert_recipients, test_alert_scope	Assignment order, factory and department matching, sources, manual send.
test_alert_buckets	Weekly versus urgent routing and the urgent dedup.
test_default_group_emails, test_production_mail	Group mailbox, always-on mailbox and Head Office routing.
test_excel_import	Parsing, preview and idempotent apply.
test_ocr, test_employee_ocr_extract	Text and scanned extraction; extract-then-confirm for documents, licenses, checkups.

Suite	Covers
`test_full_project`	End-to-end pass over the whole application.

22. Operating notes and known limits

These are behaviours of the current code that operators and maintainers should know. None stops the system working; several are deliberate trade-offs.

- **One instance only.** SQLite has one writer. Keep a single process and a single replica.
- **No idle timeout, no business audit trail.** A session ends when the browser closes. System Logs do not record who changed which field.
- **Blank dates are accepted.** A document, license or checkup saved or renewed with neither an expiry date nor the no-expiry box ticked is stored with no expiry and shows Valid.
- **Global Access is for alerts, not for viewing.** A user with no factory whose role is not Super Admin, Administration or Compliance Viewer receives alerts for their department in every factory but sees no records in the application.
- **Recipients need a department.** Because matching is by factory and department, a user without a department is never emailed for a record that has one, whatever the role.
- **Manual alerts replace assigned recipients** with the people ticked on the Send alert page.
- **Archived licenses and checkups have no restore screen**; documents do.
- **Department delete is blocked** while documents or employees use it; users assigned to it must be moved first or the database refuses the delete and the generic error page appears.
- **Abandoned import previews** stay in uploads/_imports and are included in backups. Abandoned OCR staging files are cleared at the next start after 24 hours.
- **The 'file too large' page** loads Bootstrap and a font from public CDNs, so it appears unstyled on a network without internet access. Every other page uses self-hosted assets.
- **Mail is unencrypted** on the default relay and failures are recorded as an exception class name only.
- **Kubernetes Secret.** deploy/k8s/10-secret.yaml stores SECRET_KEY in clear text. Keep the file out of shared repositories and rotate the key if it has been shared.
- **README.md** predates the Regulatory Directory, shared departments, Transport roles and the container deployment. This document and the User Manual supersede it.

23. What this edition corrects or adds

Compared with the 8 October 2026 edition, checked against the source code on 11 October 2026:

Area	8 October edition	This edition
Who may receive	A factory-scoped user only receives mail for their own factory.	A recipient must match the record's factory **and** department. Global Access (no factory) matches every factory within the user's department.
Uploader	Always included.	Included when active, with a valid email, and matching factory and department.
Group mailboxes	A department-name mailbox for Compliance or Transport records.	The mailbox is stored on the department. compliance_dept@ is copied on every alert (ALWAYS_INCLUDE_GROUP_EMAILS).
Users	Role and factory.	Role, factory (or Global Access) and department.
Test Alert	Listed as a Super Admin screen.	Removed from the application.
Administration role	Companies, departments, documents.	Views companies and departments and sets their recipients; cannot add, edit or delete them. Full document and employee rights, including archive and delete.
Urgent mail	Once per expiry date.	Once per recipient for each status (7 days, then Expired) and expiry date.
Manual alert	Same recipient rules, no dedup mark.	Group mailboxes pre-ticked; the ticked people become the document's assigned recipients.

Area	8 October edition	This edition
Employees	No delete, only archive.	Employees are set Inactive. Licenses and checkups archive, with no restore screen.
Deployment	Windows Task Scheduler only.	Also a Docker image and Kubernetes manifests (section 4).

This picture matches the application as of 11 October 2026: one Flask process, server-rendered pages, SQLite, local files, office SMTP, local OCR, and either four Windows scheduled tasks or the container deployment with three CronJobs.